

Universal Router v2.3.0 Audit



Uniswap

October 5, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	7
Nested v4 Swaps (universal-router #491 and #512)	8
Bounded Decoding of Caller-Supplied Calldata (universal-router #497 and v4-periphery #593)	8
Exact-Output Swaps in V4Router (v4-periphery #564 and #584)	9
Documented Donation Limitation (universal-router #504, in #512)	10
Execution-Time Amounts (universal-router #508 and v4-periphery #599)	11
Protocol Fee Update Commands (universal-router #509)	11
Transient Slot Table and Payments Refactor (universal-router #507 and #522)	12
Security Model and Trust Assumptions	12
Low Severity	15
L-01 Allow-Revert Sub-Plans Silently Skip the Nested Execution Gate	15
L-02 Empty-Path Swaps Succeed Without Delivering Output	15
L-03 Permit2 Transfer Commands Ignore the USE_RESOLVED_AMOUNT Sentinel	16
L-04 Sub-Plans Can Overwrite the Parent Plan's Resolved Amount	16
Notes & Additional Information	17
N-01 The Exact-Output Comment Describes an Over-Delivery That Cannot Happen	17
N-02 V2 Donation Comment Inaccuracies	18
N-03 Nested Execute Path Requires Supplying a Deadline	18
N-04 V2 Exact-Output Swaps Can Succeed Without Delivering the Requested Output	19
N-05 Fully Hook-Funded Exact-Output Routes Succeed and Leave Pre-Funded Input in the Router	19
N-06 The Head-Word Guard in toLengthOffset Wraps for a Large _arg	20
N-07 FLAG_ALLOW_REVERT Has No Effect on Some Universal Router Commands, Contrary to the README	20
N-08 Missing Head-Word Bound Check in toBytes	21
N-09 Typos and Inaccurate Documentation	21
N-10 Duplicated Transient Slot Declarations Allow the Allocation Table to Diverge	22
N-11 Redundant Interface Cast in V4_PROTOCOL_FEE_UPDATE	22
Conclusion	23
Appendix	25
Issue Classification	25

Summary

Type	DeFi	Total Issues	15 (0 resolved)
Timeline	From 2026-09-09 To 2026-09-18	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	4 (0 resolved)
		Notes & Additional Information	11 (0 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the changes that make up the Universal Router v2.3.0 release, across the [Uniswap/universal-router](#) and [Uniswap/v4-periphery](#) repositories. Two OpenZeppelin teams reviewed the pull requests, and the results are delivered as a single combined report. A follow-up engagement then reviewed the remaining changes of the release: the [diff](#) from [8b8595ae](#) to the [dev-2.3.0](#) head at commit [6c358f9d](#) and the matching v4-periphery [diff](#). Its findings were appended to this report.

In the universal-router repository:

- [pull request #491](#) at commit [8cfbb3f9](#): allows a `V4_SWAP` command to run inside a `PoolManager` unlock opened by another contract.
- [pull request #497](#) at commit [4de0e15c](#): bounds the decoding of each command's input to that input's declared length.
- [pull request #512](#) at commit [8b8595ae](#): the [dev-2.3.0](#) release branch. It gates the nested path behind an opt-in entry point ([#502](#)), documents the donation limitation of the per-hop price bound ([#504](#)), and advances the `lib/v4-periphery` submodule.
- [pull request #507](#) at commit [09b2ae2f](#): allocates transient storage slots from a compact table.
- [pull request #522](#) at commit [d1f4ce7d](#): makes the payment functions share their transfer, balance and unwrap paths, with no change in behavior.
- [pull request #508](#) at commit [51873da7](#): adds the `RESOLVE` command, which resolves amounts at execution time.
- [pull request #509](#) at commit [6c358f9d](#): adds protocol fee update commands for v4 and v3 pools.

In the v4-periphery repository:

- [pull request #564](#) at commit [df30d7eb](#): reverts exact-output swaps that are only partially filled.
- [pull request #584](#) at commit [7525d75a](#): tolerates exact-output hops whose input a hook pays in full.
- [pull request #593](#) at commit [0cc6e164](#): decodes the four v4 swap parameter structs with `abi.decode`.
- [pull request #599](#) at commit [ed72ce47](#): adds the `_mapSwapAmount` hook to `V4Router`, which maps swap amounts at execution time.

In scope were the following files:

```
(pull request #491)
universal-router
├── contracts
│   └── base
│       └── Dispatcher.sol
└──

(pull request #497)
universal-router
├── contracts
│   ├── base
│   │   └── Dispatcher.sol
│   ├── interfaces
│   │   └── IUniversalRouter.sol
│   ├── libraries
│   │   └── Commands.sol
│   └── modules
│       ├── Payments.sol
│       ├── V3ToV4Migrator.sol
│       └── uniswap
│           └── v3
│               └── BytesLib.sol
└──

(pull request #512)
universal-router
├── contracts
│   ├── UniversalRouter.sol
│   ├── base
│   │   └── Dispatcher.sol
│   ├── interfaces
│   │   └── IUniversalRouter.sol
│   ├── libraries
│   │   └── NestedUnlock.sol
│   └── modules
│       └── uniswap
│           ├── v2
│           │   └── V2SwapRouter.sol
│           └── v3
│               └── V3SwapRouter.sol
├── lib
│   └── v4-periphery (submodule, 07336f21 → 0cc6e164)
└──

(pull request #564)
v4-periphery
├── src
│   ├── V4Router.sol
│   └── interfaces
│       └── IV4Router.sol
└──

(pull request #584)
v4-periphery
├── src
│   └── V4Router.sol
└──
```

```

(pull request #593)
v4-periphery
└─ src
    ├── V4Router.sol
    └─ libraries
        ├── CalldataDecoder.sol
        └─ PathKey.sol

(pull requests #507, #508, #509 and #522)
universal-router
└─ contracts
    ├── UniversalRouter.sol
    ├── base
    │   ├── Dispatcher.sol
    │   └─ RouteSigner.sol
    ├── interfaces
    │   ├── IAmountResolver.sol
    │   └─ external
    │       ├── IV3FeeAdapter.sol
    │       └─ IV4FeeAdapter.sol
    ├── libraries
    │   ├── Commands.sol
    │   ├── Constants.sol
    │   ├── Locker.sol
    │   ├── MaxInputAmount.sol
    │   ├── NestedUnlock.sol
    │   ├── ResolvedAmount.sol
    │   └─ TransientSlots.sol
    └─ modules
        ├── Payments.sol
        └─ uniswap
            └─ v4
                └─ V4SwapRouter.sol

(pull request #599)
v4-periphery
└─ src
    └─ V4Router.sol

```

The `lib/v4-periphery` submodule pin carried by #512 is the head commit of #593, so the router and the periphery changes were reviewed together as the single binary the release produces. Both #512 and #593 were still open at the end of the first engagement. For the follow-up engagement, #507, #508, #509 and #522 had been squash-merged into `dev-2.3.0`, whose `lib/v4-periphery` pin is `ed72ce47`. The two merges from `main` in that range only touch deployment addresses and scripts and were out of scope.

System Overview

The Universal Router is the entry point through which Uniswap's front ends and integrators execute a trade. A caller submits a *plan*: a string of one-byte commands together with one ABI-encoded input per command. The router walks that plan and dispatches each command in turn. The commands cover v2, v3 and v4 swaps, Permit2 permits and transfers, wrapping and unwrapping WETH, sweeps and transfers of leftover balances, v3-to-v4 position migration, nested sub-plans, and bridge deposits. Assembling those primitives into a correct plan is the caller's job. The router itself is not designed to hold funds between transactions, has no owner and no upgrade path, and resolves every dependency from immutables fixed at deployment.

V4Router, which the Universal Router inherits from the v4-periphery repository, handles the v4 leg of a plan. A **V4_SWAP** command carries its own sub-program of v4 *actions* (swap, settle and take) that **V4Router** executes inside a **PoolManager** unlock. Within that unlock, nothing moves immediately. Each swap records a *delta* per currency, which is negative when the router owes the pool and positive when the pool owes the router. The **SETTLE** and **TAKE** actions at the end of the action list pay off and collect those balances. The unlock reverts unless every delta has returned to zero. A pool may carry a hook: arbitrary third-party code that runs before and after each swap. A hook may return deltas of its own to shift part of the cost or the proceeds of a swap onto itself.

Version 2.3.0 collects three groups of changes into that picture. The first composes a v4 swap inside an unlock that another contract already opened. The second bounds what the two routers read out of caller-supplied calldata. The third corrects how exact-output swaps in **V4Router** handle partial fills and hook-funded input. A fourth change is comment-only and records a known limitation of the per-hop price bound. The follow-up engagement covered the rest of the release. These changes add a **RESOLVE** command that fixes an amount at execution time and two permissionless commands that push protocol fees into v4 and v3 pools. They also add a compact table for the router's transient storage slots and a behavior-preserving refactor of the payment helpers.

Nested v4 Swaps (universal-router #491 and #512)

A `V4_SWAP` normally opens its own `PoolManager` unlock. Because a second unlock reverts with `AlreadyUnlocked`, a contract that already holds an unlock could not include a `V4_SWAP` in its plan. One example is a zapper that composes a router swap inside its own lock.

Pull request #491 lifted that restriction. The `V4_SWAP` branch of the dispatcher consults `poolManager.isUnlocked`. If the manager is already unlocked, the branch decodes the actions and runs them directly inside the existing lock instead of opening a new one.

Inferring consent from chain state turned out to be the weak point, and #502, which #512 carries, replaces it with an explicit opt-in. Deltas in v4 accrue under the router's own address for the lifetime of the surrounding unlock, whereas the router's reentrancy lock clears between external calls. Two unrelated callers reaching the router inside the same unlock therefore share one delta account. The second caller can pay, or collect, an unsettled balance that the first leaves. Consent now lives on the entry point: `executeNested` sets a transient flag that the dispatcher's nested branch requires, so a `V4_SWAP` that reaches that branch through plain `execute` or `executeSigned` reverts with `NestedExecutionNotPermitted` rather than nesting silently. For callers that do opt in, the sharing is accepted rather than prevented. The `IUniversalRouter` NatSpec states both directions of the exposure: a later caller can pay debt left behind and can take credit left behind.

Bounded Decoding of Caller-Supplied Calldata (universal-router #497 and v4-periphery #593)

Both routers decode their inputs in assembly. Each command input and each v4 action parameter arrives as a `bytes` slice with a declared length. Both pull requests enforce the invariant that decoding a slice reads only bytes inside it. That invariant matters most for the `executeSigned` entry point, which hashes each input at its declared length. The signature does not cover any byte read from outside a slice, so a route could be executed that the signer never approved.

Pull request #497 applies the invariant at the Universal Router's command boundary. It adds a `checkInputLength` guard ahead of the fifteen commands whose static head the dispatcher reads straight out of calldata. A sixteenth guard sits inside a new `decodePermitBatch` helper that replaces the inline Permit2 batch pointer arithmetic. This helper validates the struct offset, the details array offset and the element count against the input's length. It rewrites the

shared dynamic-array decoder `BytesLib.toLengthOffset` to take an element size and to confirm that both the offset word and the length word lie inside the slice. The rewritten decoder also bounds the element count by division so the arithmetic cannot overflow. It also adds selector and token-ID length checks to the position manager call validation. The array rewrite also supersedes the bounds fix for the medium-severity issue reported in a previous audit. In that issue, an element count was compared against a byte length, and short calldata could silently decode the per-hop price array as zeros. This could disable per-hop slippage protection. Two behavioral changes accompany the hardening: `SWEEP`'s minimum amount widens from `uint160` to `uint256`, and a new `UNWRAP_WETH_EXACT` command unwraps an exact amount of WETH rather than the router's whole balance.

Pull request #593 applies the same invariant on the v4-periphery side. The four v4 swap parameter decoders documented themselves as equivalent to `abi.decode`. However, assigning a struct pointer in assembly suppresses the compiler's validation. Solidity then bounds that struct's dynamic members against the size of the whole transaction's calldata rather than against the enclosing slice. A member could therefore begin inside the parameters and extend past them. The pull request decodes the four structs with `abi.decode` instead, which works from a copy of exactly the slice's length. The compiler therefore bounds the struct offset, the path array, each hop's hook data and the per-hop price array. The decoders now return memory rather than calldata, which propagates through `V4Router`'s swap helpers and `PathKeyLibrary.getPoolAndSwapDirection`.

The two changes meet in #512. The escape that #593 closes was only reachable through the nested path. The ordinary path passes through `PoolManager.unlock`, which re-encodes the data on its way to the callback. This leaves nothing past the end of the slice to read. In the shipped binary, it is therefore #593, brought in by the submodule bump, that keeps the nested path bounded, and the nested branch decodes straight from its calldata slice. An interim router-side fix, a self-call that re-encoded the input before decoding it, was added while #502 was being prepared. It was removed again once the new decoders made it redundant, so it does not appear in #512's diff. This dependency is also why #512 is not intended to merge before #593 does.

Exact-Output Swaps in `V4Router` (v4-periphery #564 and #584)

A v4 swap runs to the global price bound, so an exact-output swap through a pool that runs out of liquidity stops early, having delivered less than was asked for. `V4Router` bounded only the input spent, through `amountInMaximum`, and never compared the output actually

received against the output requested, so the shortfall passed unnoticed. The same omission distorted the optional per-hop price bound, which was computed from the *requested* output over the *actual* input and therefore overstated the execution price. On a multi-hop route, the shortfall could be hidden entirely. The `PoolManager` nets one delta per currency across the whole unlock, so a shortfall in a currency that appears elsewhere in the route merges into that other balance. The same shortfall was previously reported against exact-output swaps routed through the v3-family aggregator hooks, where it was fixed in the hook; #564 is the router-side generalization of that fix.

Pull request #564 makes exact output all-or-nothing. `_swap` now returns the pool's full `BalanceDelta`, and two helpers, `_swapInput` and `_swapOutput`, extract the realized legs from it. If the realized output falls short, the single-hop path and every hop of the multi-hop path revert with a new `V4ExactOutputUnfilled` error. The per-hop price bound is now computed from the realized output. Exact-input swaps are unchanged, as they already checked delivery through `amountOutMinimum`.

Pull request #584 then handles a shape that ordinary pools never produce but hooks legitimately do. A hook may pay a swap's input in full, which leaves the router holding a positive output against an input delta of exactly zero. `V4Router` had assumed that a positive output implied a strictly negative input, so such a route reverted twice over. The per-hop price bound divided by zero, and the multi-hop loop fed a zero amount to the next pool upstream, which the `PoolManager` rejects. The fix is three lines. If the input is zero, the router skips the price division, since an infinite realized price clears any finite bound. It also stops the multi-hop loop at that point, since the upstream hops have nothing left to produce. The fill check introduced by #564 runs before the zero-input branch, so a funded hop that delivers less than requested still reverts.

Documented Donation Limitation (universal-router #504, in #512)

The remaining change carried by #512 is comment-only. The per-hop price bound sizes a v2 trade from the pair's token balance in excess of its recorded reserves. The v3 path sizes a whole-balance trade from the router's own balance. In both cases, a third party can enlarge the apparent trade by transferring tokens in. The larger trade earns a worse average rate and can therefore trip the caller's per-hop bound and revert an otherwise valid route. Pull request #504 records this in `V2SwapRouter` and `V3SwapRouter`. It documents a known donation griefing attack, but the technical mitigation is deferred to a future update. The client states on the pull

request that a correct fix requires routing to emit different per-hop values, and that the encoding migration has to be decided first.

Execution-Time Amounts (universal-router #508 and v4-periphery #599)

Every amount in a plan is normally fixed off-chain when the route is built. The `RESOLVE` command (`0x15`) lets a route instead use an amount that is only known on-chain at execution time, such as a live lending-position debt. It calls `resolveAmount` on a caller-chosen resolver through `staticcall`, copies at most one word of return data, and stores the result in a transient *resolved-amount register*. A revert, a short return or a zero result counts as a failure and clears the register in the executing call frame. Whether that failure reverts the plan follows the usual `FLAG_ALLOW_REVERT` rules.

Later commands consume the value by writing the `USE_RESOLVED_AMOUNT` sentinel, the maximum `uint128` value, in their amount field. The sentinel is resolved in the amount fields of the v2 and v3 swap commands, in the `TRANSFER` value, and in the `amountIn` and `amountOut` of every v4 swap action. Slippage limits keep their literal values. A command that uses the sentinel while the register is empty reverts with `ResolvedAmountUnset`. The sentinel therefore never turns into a zero amount, which the v4 helpers would read as `OPEN_DELTA`. The v4 amounts are reached through a new virtual `_mapSwapAmount` hook in `V4Router`, which runs before the `OPEN_DELTA` check and returns the amount unchanged by default. The router's `V4SwapRouter` overrides it to read the register and cast the result to `uint128`. The register persists across the plan and any `EXECUTE_SUB_PLAN`, and is cleared when the top-level call returns.

Protocol Fee Update Commands (universal-router #509)

New pools start with no protocol fee. `V4_PROTOCOL_FEE_UPDATE` (`0x16`) and `V3_PROTOCOL_FEE_UPDATE` (`0x17`) let any route call `triggerFeeUpdate` on the governance fee adapter. The fee can then be set in the same transaction as the pool's first swap, instead of waiting for the off-chain keeper. The router stores no adapter address. The v4 command reads `protocolFeeController` from the `PoolManager`, and the v3 command reads the `owner` of the v3 factory, on every call. A revert from the adapter follows `FLAG_ALLOW_REVERT`, but the lookup itself does not. On a chain where the `PoolManager` or the v3 factory is not deployed, the whole call reverts.

Transient Slot Table and Payments Refactor (universal-router #507 and #522)

Pull request #507 replaces the router's keccak-derived transient storage slots with a single `TransientSlots` table of small literals (1 to 7), saving roughly 700 bytes of bytecode. `Locker`, `MaxInputAmount`, `RouteSigner`, `NestedUnlock` and `ResolvedAmount` repeat their slot numbers as literals inside their assembly blocks, so each must match the table. New slots may only be appended, and no contract in the inheritance tree may declare a Solidity `transient` state variable, since the compiler would allocate it from slot 0 upward. Pull request #522 moves the repeated transfer, balance-lookup and unwrap logic of `Payments` into shared private helpers and is not meant to change behavior.

Security Model and Trust Assumptions

The Universal Router and `V4Router` define no privileged roles. There is no owner, no pause, no upgrade path and no rescue function. Every dependency is an immutable fixed at deployment and is trusted to behave as specified. These dependencies are the `PoolManager`, Permit2, WETH9, the v2 and v3 factories and their pool init code hashes, and the v3 and v4 position managers. They also include the permissions adapter factory and the Across spoke pool. The only exception is the protocol fee adapters, which are looked up through the `PoolManager` and the v3 factory at call time. The trust boundaries that matter run between a caller and the plan it submits, the resolvers and pool hooks that plan touches, and any other party in the same transaction.

Callers own their plans. Every parameter that decides the outcome of an execution comes from the caller: the commands, their inputs, the pools, the recipients, the payer flags and the slippage bounds. `amountOutMinimum`, `amountInMaximum` and the optional per-hop price bounds are the caller's protection against unfavorable execution, and each can be disabled by passing zero or an extreme value. Plans are normally built off-chain by the Uniswap SDKs; the decoding guards added in this release bound what the router reads, but they cannot make a badly constructed plan safe.

The router is a pass-through, and residues are unowned. The router is expected to hold nothing once a plan finishes. Anything a plan leaves behind sits at an address whose `SWEEP` and `TRANSFER` commands are callable by anyone, so it belongs to whoever executes next.

Examples are an exact-unwrap remainder, the unspent input of an exact-output swap, or a native balance forwarded and not consumed. A plan that pre-funds the router must therefore end by returning the unspent balance. Two changes in this release make that responsibility more visible. First, `UNWRAP_WETH_EXACT` is designed to leave the remainder behind. Second, with the early exit that #584 adds, a route whose input a hook funds entirely can now succeed while consuming nothing. The whole pre-funded amount then stays in the router, where the plan previously reverted.

Hooks are untrusted. Any account can deploy a hook and initialize a pool with it, and a plan may name any pool. Hook code therefore runs inside the caller's unlock, can revert, and can return deltas that change the input and output of a swap. The router accounts for hook deltas but does not police them. It rejects only the shapes it cannot represent: a hook that funds more than the whole input or takes more than the whole output fails on a safe cast. On an exact-output request, a hook may also raise the amount the pool is asked to swap and keep the difference. The caller's own `amountInMaximum` and per-hop bounds are then the operative limits. Hooks reading the signed-route context exposed by `executeSigned` must check both that the caller is the `PoolManager` and that the swap's sender is the Universal Router. Otherwise, another contract in the execution chain can start its own swap under someone else's signed context. Pull request #497 corrects this requirement in the interface documentation.

Nesting shares the router's delta account, by design. Calling `executeNested` explicitly accepts that deltas and credits left under the router's address persist for the lifetime of the surrounding unlock. Any other call reaching the router within that unlock can pay or collect them. The router does not re-assert a per-plan delta checkpoint, so a contract that opts in and exposes a permissionless path to that entry point can have its own capital drained. The documented integrator pattern is that a function which does not open its own unlock must call `execute` instead.

Signed routes commit to inputs, not to the surrounding calldata. `executeSigned` hashes the commands and each input at its declared length. The bytes around those inputs, such as non-canonical outer encodings or trailing data, are chosen freely by whoever submits the transaction. The decoding work in #497 and #593 exists to uphold this property. It is what makes a decoder reading past a slice a signature-bypass concern rather than a caller-only one.

Resolvers are chosen by the route builder. The resolver address and its context are part of the plan, and the router does not check that a resolver is honest. A resolver only decides when an amount is fixed. The value it returns is still limited by the checks of the command that consumes it, such as Permit2 allowances, `amountInMax`, `amountOutMin` and the router's actual balance for `TRANSFER`. Callers are trusted to pick resolvers whose results they accept

and to set slippage limits that stay meaningful when the amount is only known at execution time. This matters most for exact-input swaps, where `amountOutMin` stays a literal while the input varies. A resolver that consumes all the forwarded gas can only make the caller's own transaction fail. The maximum `uint128` value is now the sentinel, so it can no longer be used as a literal amount in the fields that map it. No existing integration is assumed to rely on passing it.

The resolved-amount register is shared across a plan. Only `RESOLVE` is assumed to write the register, and it is only read through the sentinel. The register is not scoped to a call frame, so a sub-plan sees, and can overwrite, the value its parent resolved. A sub-plan that reverts under `FLAG_ALLOW_REVERT` also rolls back its own writes to the register, including the clearing write of a failed `RESOLVE`. Whether the register is meant to be scoped to a plan or shared across sub-plans is not documented.

Protocol fee adapters are trusted. The fee update commands call whatever contract is currently registered as the v4 `protocolFeeController` or the v3 factory `owner`. Governance installs these contracts, and they are trusted to be permissionless and non-malicious. The call happens partway through a plan, while the router may temporarily hold user funds. The reentrancy lock blocks re-entry into `execute`, but the adapter still runs arbitrary code with the router as `msg.sender`. The commands are expected to be used only on chains where the matching `PoolManager` or v3 factory is deployed.

Transient slots must stay unique. Every slot number repeated inline is assumed to match its `TransientSlots` entry, and no two features are assumed to share a slot. No inherited contract or dependency is assumed to use transient storage in the router's context. The slot reallocation and the `Payments` refactor are assumed not to change any externally visible behavior apart from the new slot numbers.

The release is close to the contract size limit. `UniversalRouter.sol` is pinned to `optimizer_runs = 1` in `foundry.toml`, and even so its runtime code measures 24,353 bytes against the EIP-170 limit of 24,576, leaving 223 bytes. Any fix made on the router side has very little room, which constrains where a change can be made.

Token assumptions. Standard ERC-20 tokens and native ETH are supported. Fee-on-transfer tokens are only partially accommodated. The v2 exact-input path measures the recipient's balance change and so enforces `amountOutMinimum` on the amount that actually arrives. In contrast, the v2 exact-output path sizes the trade from a quote and never verifies delivery.

Low Severity

L-01 Allow-Revert Sub-Plans Silently Skip the Nested Execution Gate

The `EXECUTE_SUB_PLAN` command reaches a sub-plan through a low-level self-call, so a failure is captured into the `success` flag rather than propagated. The `execute` function then discards that captured failure whenever the command byte carries the `FLAG_ALLOW_REVERT` bit, as determined by `successRequired`. Wrapping a `V4_SWAP` one level down therefore converts the `nested execution gate` from a hard revert into a silent skip.

A contract holding a `PoolManager` unlock open can call a victim's permissionless entrypoint and force the nested leg to become a no-op, leaving the route's input in the router where `sweep` is callable by any account. The exposure is nevertheless limited, since the same route strands the same input at the top level upon an ordinary slippage failure, and the [documented guidance for the allow-revert flag](#) already instructs integrators to include further commands that remove funds left unused in the router whenever it is set. However, the gate renders that failure triggerable by an unrelated party at no cost.

Consider re-throwing `NestedExecutionNotPermitted` when it is the captured output of a sub-plan, so that the gate cannot be reduced to a silent skip. Alternatively, if leaving the gate skippable through the allow-revert flag is the intended behavior, consider documenting it alongside the [existing allow-revert guidance](#), making explicit that a `V4_SWAP` nested under an allow-revert sub-plan can be turned into a no-op by an unrelated party at will, and that the cleanup commands that recover the unused input are therefore mandatory rather than advisory for such routes.

L-02 Empty-Path Swaps Succeed Without Delivering Output

`V4Router._swapExactInput()` and `V4Router._swapExactOutput()` iterate over the provided swap path without rejecting an empty one. For exact-input swaps, an empty path leaves `amountOut` at zero and succeeds when `amountOutMinimum` is zero. For exact-output swaps, it leaves `amountIn` at zero and succeeds after checking only

`amountInMaximum`, without verifying the requested output because no hop is executed. In both cases, the call can therefore complete without performing a swap.

A malformed route can be reported as successful even though the caller's input and output balances remain unchanged. This cannot leave the PoolManager short, since any debt in the output currency the swap was meant to cover stays open and the unlock reverts with `CurrencyNotSettled` unless another action pays it, but integrations that treat successful execution as proof of fulfillment may incorrectly record a swap as completed.

Consider rejecting empty paths with a dedicated error before entering either loop, and adding a regression test for each loop that an empty path reverts.

L-03 Permit2 Transfer Commands Ignore the `USE_RESOLVED_AMOUNT` Sentinel

The [USE_RESOLVED_AMOUNT documentation](#) states that amount fields are mapped through the resolved-amount register. However, the `amount` of `PERMIT2_TRANSFER_FROM` and the amounts in `PERMIT2_TRANSFER_FROM_BATCH` are forwarded to Permit2 unchanged, so neither command can consume an amount produced by `RESOLVE`. A route that resolves an amount onchain must therefore pull an upper bound guessed offchain and sweep the remainder back. If a route places the sentinel in these fields, Permit2 receives the literal `type(uint128).max`.

Consider mapping the `amount` of `PERMIT2_TRANSFER_FROM` through `ResolvedAmount.map` and casting the result with `toUint160`. This is the path a mapped `amountIn` already takes through `payOrPermit2Transfer` when the user pays for a v2 or v3 swap. The sentinel does not collide with Permit2, whose only special amount is a `type(uint160).max` allowance.

L-04 Sub-Plans Can Overwrite the Parent Plan's Resolved Amount

The value stored by `RESOLVE` lives in a single register shared by a plan and all of its sub-plans. A `RESOLVE` inside a sub-plan overwrites the value set by the parent plan. The new value remains after the sub-plan returns, because the register is only [cleared when the top-level execute exits](#). Any later command in the parent plan that uses `USE_RESOLVED_AMOUNT` therefore reads the sub-plan's value instead of the one the parent resolved.

For example, consider the following route:

1. The parent plan calls `RESOLVE`, which returns the user's debt of 100 USDC.
2. The parent plan calls `EXECUTE_SUB_PLAN`. The sub-plan calls its own `RESOLVE` for an amount it needs internally, which returns 150 USDC.
3. The parent plan calls `TRANSFER` with `USE_RESOLVED_AMOUNT`, intending to send the 100 USDC debt. It sends 150 USDC instead.

The reverse also happens. If a sub-plan with `FLAG_ALLOW_REVERT` resolves an amount, or fails to resolve one, and then reverts, its writes to the register are rolled back. The parent then consumes its own earlier value. If the register is meant to be shared, this contradicts the [_resolve documentation](#), which says every failure clears the register. The documentation does not say which of the two behaviors is intended. To know which value a later command will use, the author of the parent plan must inspect every sub-plan and work out whether it could revert.

Consider documenting whether resolved amounts are scoped to each plan or shared across sub-plans. If they are scoped, consider restoring the register to its previous value when `EXECUTE_SUB_PLAN` returns, whether or not the sub-plan reverted. If they are shared, consider clearing the register when an allowed-revert sub-plan fails.

Notes & Additional Information

N-01 The Exact-Output Comment Describes an Over-Delivery That Cannot Happen

The [#564](#)'s new comment in `_swapExactOutputSingle` says [over-delivery is "possible only via hook pools" and allowed](#). That state is not reachable. A `beforeSwap` hook can only [add its specified delta to the amount the pool is asked for](#), the pool [returns what it actually filled](#), capped by `SwapMath` at the requested amount, and the hook's delta is then [subtracted back out](#). The `afterSwap` return delta only touches the unspecified side. The router's output leg is therefore always the requested amount minus any unfilled remainder, never more, so the [≤ check](#) is purely a shortfall guard. This has no security impact, but the comment states a property the code does not have.

Consider rewording the comment to say that an exact-output swap returns `amountOutActual` at most `amountOut` and that the check rejects a shortfall.

N-02 V2 Donation Comment Inaccuracies

The [donation note in V2SwapRouter](#) states that a donation "drives the price below `minHopPriceX36` and reverts". That revert is not always triggered. It occurs only when the donation is large enough to [push the measured price under the bound](#). Furthermore, the per-hop check may not be active at all, as it is disabled when [the array is empty](#) and when [the individual entry is zero](#). In those cases, the donation is swapped into the trade and delivered to the route's recipient.

The note further states that the pair's permissionless `skim()` "then returns the donation", which can be read as returning the excess to the original donor. However, [skim\(address to\)](#) forwards the pair's whole balance excess to a caller-specified destination. The excess therefore reaches whichever account calls it first, which is not necessarily the donor.

Consider rewording the note so that the per-hop bound is described as one a donation can trip, matching the wording already used in the [sibling note in V3SwapRouter](#), and so that `skim` is described as forwarding the excess to a caller-specified recipient rather than returning it.

N-03 Nested Execute Path Requires Supplying a Deadline

The [executeNested function](#) is declared only as `executeNested(bytes,bytes[],uint256)`, whereas `execute` is available both [with a deadline](#) and [without one](#). A composer that calls the two-argument `execute` and now requires the nested path must therefore source a deadline.

Consider adding the two-argument overload for parity with `execute`, or documenting that requiring the three-argument form is intentional.

N-04 V2 Exact-Output Swaps Can Succeed Without Delivering the Requested Output

`V2SwapRouter.v2SwapExactOutput()` calculates the nominal input required for `amountOut`, checks it against `amountInMaximum`, transfers it to the first pair, and executes the path without measuring the recipient's final output balance. However, `_v2Swap()` calculates each hop's output from the input tokens actually received by the pair. If the input token charges a transfer fee, the pair receives less than the nominal `amountIn` and delivers less than the requested `amountOut`, while the exact-output command still succeeds.

For example, with an input token that charges a 10% transfer fee, a swap requesting 100 ether of output completes successfully while the recipient receives roughly 90 ether. The optional per-hop price checks do not guarantee the declared output amount because they only compare the realized price against a price floor. Integrations that treat successful execution as proof of fulfillment may therefore settle an underdelivered operation.

Consider measuring the recipient's output-token balance change and reverting unless it is at least `amountOut`, similarly to [the exact-input path](#). If fee-on-transfer tokens are intentionally unsupported for exact-output swaps, consider documenting and enforcing that restriction instead of allowing silent underdelivery.

Note: This issue is not in the audited PRs. `V2SwapRouter.sol` is unchanged by the PRs in scope; it is reported as a pre-existing observation on the combined state.

N-05 Fully Hook-Funded Exact-Output Routes Succeed and Leave Pre-Funded Input in the Router

In `_swapExactOutput`, an exact-output hop whose input a hook fully funds ends with `amountIn == 0` and [breaks out of the loop](#) instead of reverting `SwapAmountCannotBeZero` on the next hop. The route's input currency then carries no delta, so a `SETTLE(currencyIn, OPEN_DELTA)` resolves a zero debt and `_settle` [returns early](#) without moving anything. Any input an earlier command placed in the router, such as a `PERMIT2_TRANSFER_FROM` to `ADDRESS_THIS`, a `WRAP_ETH`, or `msg.value` for a native input, stays there unless the plan ends with a refund, and `sweep` is callable by anyone. The user is not worse off than with the same plan through an ordinary pool, where the unused remainder strands the same way, and a hook that funds the hop pays its real input cost, so

there is no profit in mounting this deliberately. What changes is the failure mode: a multi-hop plan that previously reverted now succeeds with the balance left behind.

In practice, router-funded exact-output plans end with a full-balance return of the input currency, so this is a behaviour change to record rather than a live exposure.

Consider adding a comment that a route costing zero leaves any pre-funded input in the router, so any plan with `payerIsUser = false` or a native input must end with a full-balance return of the input currency.

Note: The router now executes a fully funded multi-hop route, but [V4Quoter](#) still rejects with [SwapAmountCannotBeZero](#). The fix is in PR [#588](#). It's recommended to confirm [#588](#) ships with or before the release carrying [#584](#).

N-06 The Head-Word Guard in `toLengthOffset` Wraps for a Large `_arg`

The length guard in `toLengthOffset` checks that head word `_arg` lies inside the slice with `gt(add(shl(5, _arg), 0x20), _bytes.length)`. Both the shift and the addition can wrap modulo `2**256`, so for values such as `2**251 - 1` or `type(uint256).max` the head offset wraps to zero and the guard passes for a slice of any length. The head word is then read from outside the slice, and with a slice shorter than 32 bytes the `second guard` and the `element bound` both underflow as well, since each relies on the first guard having established `_bytes.length >= 32`.

Consider comparing in word units with division that cannot wrap: revert on `iszero(gt(div(_bytes.length, 0x20), _arg))`, i.e. unless the slice holds more than `_arg` whole words.

N-07 `FLAG_ALLOW_REVERT` Has No Effect on Some Universal Router Commands, Contrary to the README

The [README](#) of Universal Router states that when the `f` bit of a command byte is set and the command reverts, "the transaction will continue". The router only consults that flag [after dispatch returns](#) with `success == false`, which only a few branches can return, mainly those built on a low-level `.call` such as `PERMIT2_PERMIT`, `V4_INITIALIZE_POOL` and `EXECUTE_SUB_PLAN`. Every other command, including all v2, v3 and `v4 swaps`, `SWEEP`,

`TRANSFER` and the WETH commands, reverts inside `dispatch`, so the failure unwinds `execute` before `successRequired` is reached and the flag is inert.

Consider documenting in the README which commands the `FLAG_ALLOW_REVERT` flag applies to, and stating that other commands revert regardless of the flag.

N-08 Missing Head-Word Bound Check in `toBytes`

`CalldataDecoder.toBytes` loads head word `_arg` at `_bytes.offset + 32 * _arg` without checking that the slice contains it. Its only guard is the `tail check`, which a slice of one zero word satisfies for any `_arg`. The five PositionManager decoders, `decodeModifyLiquidityParams`, `decodeIncreaseLiquidityFromDeltasParams`, `decodeMintParams`, `decodeMintFromDeltasParams` and `decodeBurnParams`, rely on `toBytes` both to read `hookData` and as their only length check.

Consider adding the head-word check to `toBytes`:

```
if iszero(gt(div(_bytes.length, 0x20), _arg)) , and reverting
SliceOutOfBounds if the check fails.
```

N-09 Typos and Inaccurate Documentation

The codebase contains multiple instances of typos and inaccurate documentation:

- The `MAX_AMOUNT_IN_SLOT comment` reads "the the" instead of "the".
- The `command type comment in Commands.sol` states that the maximum supported command is `0x3f`, while `ACROSS_V4_DEPOSIT_V3` is defined as `0x40` and routed by the `Dispatcher` contract. The comment predates the audited changes, which only touched the file to add `0x16` and `0x17`, but it no longer reflects the real bound. Given the reserved third-party range, the actual maximum is `0x5f`.

Consider correcting the typo and the documentation listed above, either updating the stale comment to the real bound or removing it, to improve the clarity and accuracy of the codebase.

N-10 Duplicated Transient Slot Declarations Allow the Allocation Table to Diverge

The `TransientSlots` library is documented as the single allocation table for every transient storage slot used by the router. However, each component that reads or writes transient storage redeclares its slot as a separate literal constant: `Locker`, `MaxInputAmount`, `NestedUnlock`, `ResolvedAmount`, and `RouteSigner`. The `tstore` and `tload` operations use these copies, not the table. Each copy carries a comment stating that it must equal the corresponding table entry, and only tests enforce the equality. Keeping two declarations of the same value in sync by hand creates room for error. For example, a future change could update or add a slot in one place but not the other. The test suite would only catch this if it were also updated.

The stated reason for the duplication is that inline assembly only accepts literal constants. While inline assembly cannot reference a library member such as `TransientSlots.LOCKER`, it does accept file-level constants with literal values, including imported ones.

Consider declaring the transient slots as file-level constants in `TransientSlots.sol` and importing them wherever they are needed. This would make the table the single source of truth for every slot and remove the need for the per-library copies and the tests that pin them. Since constants are inlined at compile time, this change does not affect the compiled bytecode.

N-11 Redundant Interface Cast in `V4_PROTOCOL_FEE_UPDATE`

The `V4_PROTOCOL_FEE_UPDATE` branch casts the `poolManager` immutable to `address` and back to `IProtocolFees` before reading `protocolFeeController`. However, `IPoolManager` already inherits `IProtocolFees`, so the function is directly reachable on `poolManager`, and the `IProtocolFees` import exists only to support the cast.

Consider calling `poolManager.protocolFeeController()` directly and removing the unused import.

Conclusion

The audited pull requests make up the Universal Router v2.3.0 release. They let a v4 swap run inside a `PoolManager` unlock that another contract has already opened, and they place that capability behind an explicit opt-in. They also bound what both routers read out of caller-supplied calldata. Exact-output swaps in `V4Router` now revert on a partial fill but still accept input that a hook funds in full. A follow-up engagement covered the rest of the release. These changes add the `RESOLVE` command, two permissionless protocol fee update commands, a compact transient slot table and a refactor of the payment helpers. Four low-severity issues and several notes were reported across both engagements. No critical, high, or medium severity issues were identified.

The two decoding changes close what they target: `universal-router#497` hardens the Universal Router's hand-written decoders, and `v4-periphery#593` replaces `v4-periphery's` with `abi.decode`. One guard that `#497` adds, the head-word check in `toLengthOffset`, can wrap for a very large element index. Because every call site passes a constant today, this is reported as a note together with a wrap-free form of the check. The one decoder left untouched, `CalldataDecoder.toBytes`, does not check that the offset word it reads lies inside its slice. Because every path that reaches it today is bounded by other means, this is reported as a note. However, a single check inside `toBytes` would cover all of its call sites at once.

The exact-output work is sound. Pull requests `v4-periphery#564` and `#584` are best read as one change. The first makes a shortfall revert, and the second readmits the one legitimate shape: a hop whose input a hook pays in full. Two behaviors on that path are worth recording. An empty swap path succeeds without swapping, and a route whose cost a hook covers entirely now succeeds while consuming nothing, leaving any pre-funded input in the router. Together with the v2 exact-output path, which never verifies delivery, this means that a successful call is not by itself evidence that a trade happened. This matters for integrations that settle on that signal.

The nesting change moves the decision from chain state onto a dedicated entry point, so a caller now shares the router's delta account only by opting in. The interface documentation states the exposure that remains for those callers. One gap remains: a `V4_SWAP` nested in a sub-plan whose command byte carries the allow-revert flag turns the gate's revert into a silent skip. An unrelated party can therefore reduce a victim's nested leg to a no-op.

The **RESOLVE** command keeps the resolver read-only, bounds the data it copies and rejects a zero result, so the sentinel cannot fall through to **OPEN_DELTA**. The register that holds the resolved value is the one underspecified part of the design. It is shared between a plan and its sub-plans, so a sub-plan can overwrite the parent's value. A sub-plan that reverts under the allow-revert flag also rolls back the clearing write of a failed **RESOLVE**, leaving the parent with an earlier value. Because the fix differs depending on whether the register is meant to be scoped to a plan or shared, settling that question is the main item to address before release. The Permit2 transfer commands also forward their amounts without mapping the sentinel, which the Uniswap team confirmed was unintended. The remaining notes concern maintainability, chiefly slot literals that are repeated outside the table they are meant to come from. The tests for **RESOLVE** cover single-hop v4 swaps and transfers only, and extending them to the multi-hop v4, v2 and v3 paths and to the nested entry point is advisable.

The Uniswap team was responsive throughout these engagements and documented each change thoroughly in its pull request, which made for an effective review.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.